Abstraction

Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Abstraction lets you focus on what the object does instead of how it does it.

Abstract class in java

- Data **abstraction** is the process of hiding certain details and showing only essential information to the user.
- Abstraction can be achieved with either **abstract**
- **classes** or **interfaces** (which you will learn more about in the next chapter).

- **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

- **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).
  - It can have abstract and non-abstract methods.

Point that to be remember

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Example 1

```
abstract class Bike{
  abstract void run();
  }
 class Honda4 extends Bike{
void run()
{
System.out.println("running safely");
}
public static void main(String args[]){
Bike obj = new Honda4();
obj.run();
}
}
```

Bank example

```
abstract class Bank{
abstract int getRateOfInterest();
}
class SBI extends Bank{
int getRateOfInterest(){return 7;}
}
class PNB extends Bank{
int getRateOfInterest(){return 8;}
}

class TestBank{
public static void main(String args[]){
Bank b;
b=new SBI();
System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");

b=new PNB();
System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");

}}
```

Abstract class having constructor, data member and methods

```
abstract class Bike{
   Bike(){System.out.println("bike is created");}
   abstract void run();
   void changeGear()
{System.out.println("gear changed");}
 }
//Creating a Child class which inherits Abstract class
 class Honda extends Bike{
 void run()
 {
System.out.println("running safely..");
}
 }
//Creating a Test class which calls abstract and non-abstract methods
 class TestAbstraction2{
 public static void main(String args[]){
  Bike obj = new Honda();
  obj.run();
  obj.changeGear();
 }
}
```

Write a Java program to create an abstract class Animal with an abstract method called sound(). Create subclasses Lion and Tiger that extend the Animal class and implement the sound() method to make a specific sound for each animal.

```
/ Animal.java // Define an abstract class named Animal
abstract class Animal { // Declare an abstract method named sound
public abstract void sound();
 }
```

```
class Lion extends Animal { // Override the abstract method sound
from the Animal class
@Override
public void sound() {
System.out.println("Lion roars!"); }
 }
```

```
class Tiger extends Animal {
// Override the abstract method sound from the Animal class
@Override public void sound()
{
System.out.println("Tiger growls!");
}
}
```